Heart-Disease Risk Prediction

Building and evaluating a reproducible machine-learning service, from data preparation to deployment and monitoring.

Course	MLOps (AIMLCZG523) - Assignment 01
Dataset	Heart Disease UCI (Cleveland), 303 patients, 13 features + binary target
Student name	Dinesh Sai Teja Paruchuri
Roll number	2025AE05145
Code repository	https://github.com/dineshsaitejaparuchuri/mlops-assignment
Best model	Logistic Regression (C=0.3), registered as heart-disease-classifier v2
Held-out test	ROC-AUC 0.970; recall 1.000 at threshold 0.26 (accuracy 0.902, recall 0.929 at 0.5)

Contents

- 1. Setup and installation
- 2. Data acquisition and EDA
- 3. Feature engineering and modelling
- 4. Experiment tracking with MLflow
- 5. Architecture
- 6. CI/CD pipeline and testing
- 7. Containerisation
- 8. Kubernetes deployment and monitoring
- 9. Results and limitations
- 10. Repository and deliverables map
- 11. Evidence screenshots

Scope and verification. The results and screenshots in this report come from the project artifacts and captured runs. Docker, a local Kubernetes cluster, Prometheus and Grafana were exercised locally. GitHub Actions run #4 passed the full pipeline, including a disposable kind cluster and routed Ingress smoke test. Run #5 repeated the workflow successfully on a subsequent documentation-only commit. The local MLflow run store is generated during training and is not included in the source archive, so the report builder can also read the committed experiment summary on a clean checkout.

1. Setup and installation

I used Python 3.11 for the project. The core API, training code and tests install from `requirements.txt`. Notebook and report tools are listed separately in `requirements-notebooks.txt`, which keeps the CI install smaller.

Clean-machine setup

```
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

```
pytest test -v && flake8 src test
```

The cleaned CSV is included, so a first run does not need to download the data again. To reproduce that step, run `python -m src.download_data`; create the EDA figures with `python -m src.eda` and rerun the nine model configurations with `python -m src.train`. Notebook/report tools are installed separately from `requirements-notebooks.txt`. I fixed the random seed at 42 to make the split and training results repeatable in the same software environment. Start the service locally with `uvicorn src.api:app --port 8000`.

Repository layout

Path	Purpose
src/download_data.py, eda.py	data acquisition + cleaning, EDA figures and summary
src/features.py	feature engineering and the scikit-learn preprocessing pipeline
src/train.py	model search, MLflow logging, registry, threshold selection, export
src/api.py, schemas.py	FastAPI service: /predict, /health, /metrics, request logging
test/	18 pytest tests (features, API, model artifact round trip)
notebooks/	01_eda, 02_training, 03_inference - executed, outputs saved
Dockerfile, docker-compose.yml	multi-stage image; compose adds Prometheus + Grafana
k8s/, monitoring/	Kubernetes manifests, Prometheus config, Grafana dashboard
.github/workflows/ci.yml	lint, test, train, Docker smoke test, kind Ingress smoke test, artifacts
docs/	experiment record, run outputs, this report, demo script

Notebooks are regenerated and executed with `python scripts/build_notebooks.py`; they read from `data/` and `models/` only and never overwrite the trained model or the MLflow store.

2. Data acquisition and exploratory data analysis

Source. The Cleveland subset of the UCI Heart Disease data (`processed.cleveland.data`) is fetched by `src/download_data.py`. The script replaces '?' with NaN, casts every column to numeric and binarises the 0-4 severity label (>0 = disease), then writes `data/heart_cleaned.csv`: 303 rows x 14 columns (13 features + *target*).

Data quality. There are 6 missing values: ca (4), thal (2). I kept them as missing in the CSV and handle them inside the model pipeline. That way, imputation values are learned from each training fold rather than from the full dataset. The classes are reasonably balanced (164 without the target condition and 139 with it; 45.9% positive), so I did not apply resampling.

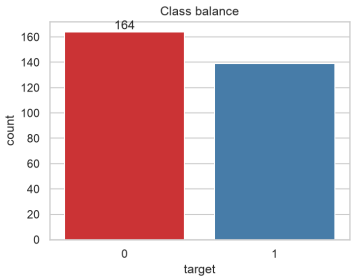


Figure 1 - class balance (screenshots/eda/class_balance.png)

Initial patterns. In this sample, the features with the strongest correlations with the target are thal (+0.53), ca (+0.46), exang (+0.43), oldpeak (+0.42), cp (+0.41). Fasting blood sugar (+0.03) and cholesterol (+0.09) have weaker linear correlations, while maximum heart rate is negatively correlated (-0.42). The highest cholesterol value is 564 mg/dl; I left it unchanged rather than assume it was an error. These are associations in a small dataset, not evidence that any one feature causes heart disease.

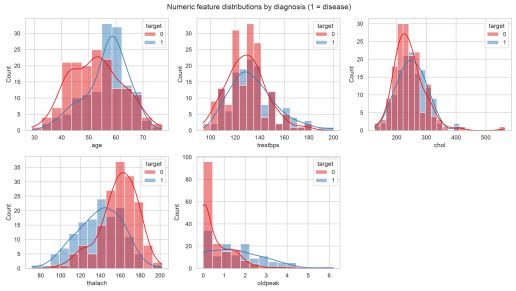


Figure 2 - numeric feature distributions by diagnosis (histograms with KDE)

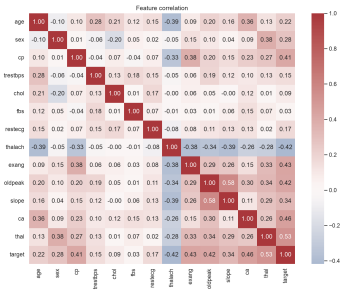


Figure 3 - correlation heatmap of all 14 columns

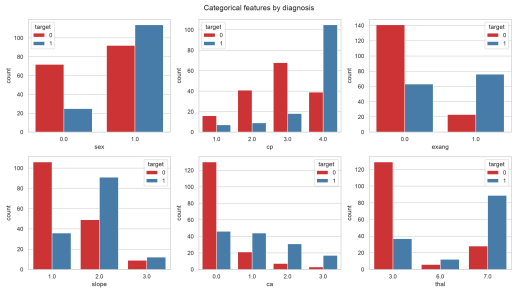


Figure 4 - categorical features by diagnosis (sex, cp, exang, slope, ca, thal)

What I took from the plots. The disease-positive group tends to be older, has lower peak heart rate and shows higher ST depression. Asymptomatic chest-pain records, exercise-induced angina, thal=7 and non-zero ca values also appear more often in that group. I used these patterns to understand the data, not to make clinical claims. The notebook and `docs/eda_summary.txt` contain the plots and summary statistics for reference.

3. Feature engineering and modelling

Feature engineering

I put preprocessing and classification in the same scikit-learn `Pipeline` (`src/features.py`) and save that complete pipeline with the model. This avoids a common deployment bug: the API cannot accidentally use different transformations from the ones used during training.

Group	Columns	Treatment
Numeric	age, trestbps, chol, thalach, oldpeak, hr_reserve	median imputation + StandardScaler
Categorical	cp, restecg, slope, thal, ca	most-frequent imputation + one-hot (unknown categories ignored)
Binary	sex, fbs, exang	most-frequent imputation, passed through

Engineered feature. `hr_reserve = thalach / (220 - age)` is the fraction of the age-predicted maximum heart rate reached during the stress test. Raw peak heart rate falls with age, so the ratio separates 'low for their age' from 'low because old'. It is added by a `FunctionTransformer`, making it part of the pipeline and therefore of the API.

Model development and tuning

I split the data into training and test sets using a stratified 80/20 split and seed 42 (242 training rows and 61 test rows). I kept the test split aside for the final report and compared two model families using small parameter grids. With only about 240 training records, a very large search could end up tuning to noise rather than a reliable pattern:

- **Logistic Regression** (max_iter 2000): C in {0.03, 0.3, 3} - 3 runs.
- **Random Forest** (300 trees): max_depth in {3, 6, None} x min_samples_leaf in {1, 4} - 6 runs.

I evaluated every configuration with stratified five-fold cross-validation on the training data, tracking accuracy, precision, recall and ROC-AUC. The top mean ROC-AUC came from **logreg-C0.3** (0.904; accuracy 0.847, precision 0.877, recall 0.783). Logistic Regression edged out the Random Forest configurations in this run. That does not mean it will always be better, but it is a sensible choice here: it performed well and is relatively light to serve and explain.

Decision threshold chosen for recall

For this exercise I preferred a threshold that favours recall, because a false negative can be more costly than an additional follow-up. Using out-of-fold predictions from the training split, I selected the highest threshold in [0.20, 0.70] that still reached 90% recall: **0.26**. The test set was not used to choose it. The held-out results show the trade-off:

held-out test (61 rows)	accuracy	precision	recall	ROC-AUC
threshold 0.5	0.902	0.867	0.929	0.970
threshold 0.26	0.836	0.737	1.000	0.970

On this particular test split, the lower threshold raised recall to 100% but reduced precision, so more healthy patients would be flagged for follow-up. There are only 61 test records; a single patient changes a metric by roughly 1.6 percentage points. I would treat these numbers as an illustration of the threshold trade-off, not as a reliable estimate of real-world clinical performance.

4. Experiment tracking with MLflow

In `src/train.py`, I log each of the nine model configurations to the local MLflow experiment `heart-disease-risk`. A run records its settings, cross-validation and test metrics, ROC/confusion-matrix plots, and fitted pipeline. The best run is tagged `selected=true`; it also receives the tuned-threshold metrics and is registered as `heart-disease-classifier` version 2.

The table below is sorted by mean cross-validation ROC-AUC. When the local MLflow store is present, the report reads metrics from it; on a clean checkout it uses the committed experiment summary so the PDF can still be rebuilt before the next training run. There are 9 recorded configurations.

run	cv acc	cv prec	cv recall	cv ROC-AUC	test ROC-AUC	run id
logreg-C0.3	0.847	0.877	0.783	0.904 ± 0.016	0.970	a7a6cacd
logreg-C3.0	0.847	0.877	0.783	0.902 ± 0.014	0.963	
random_forest-max_depth3-min_samples_leaf4	0.822	0.849	0.755	0.895 ± 0.023	0.950	
random_forest-max_depthNone-min_samples_leaf4	0.826	0.846	0.774	0.894 ± 0.025	0.947	
random_forest-max_depth6-min_samples_leaf4	0.830	0.847	0.783	0.894 ± 0.026	0.946	
random_forest-max_depth3-min_samples_leaf1	0.826	0.860	0.755	0.893 ± 0.024	0.947	
logreg-C0.03	0.814	0.866	0.711	0.888 ± 0.023	0.964	
random_forest-max_depth6-min_samples_leaf1	0.822	0.845	0.764	0.879 ± 0.026	0.949	
random_forest-max_depthNone-min_samples_leaf1	0.814	0.810	0.783	0.876 ± 0.037	0.948	

The cross-validation standard deviations (about 0.014 to 0.037 for ROC-AUC) are large enough that small gaps between runs should not be overinterpreted. In this experiment Logistic Regression was competitive with, and slightly ahead of, the Random Forest candidates. The experiment notes and metrics file are kept alongside the code so the reported figures can be checked.

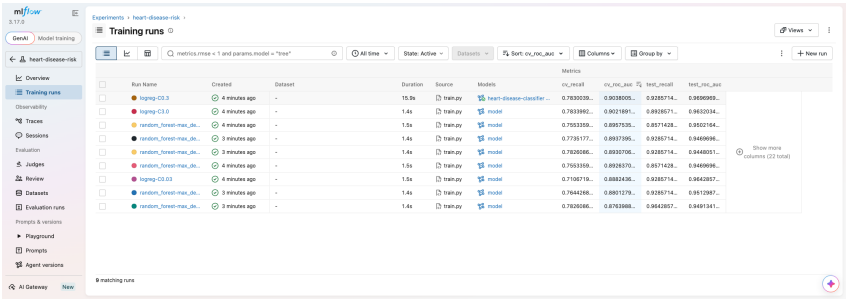


Figure 5 - MLflow UI: training runs, sorted by cv_roc_auc (real screenshot of the local MLflow server)

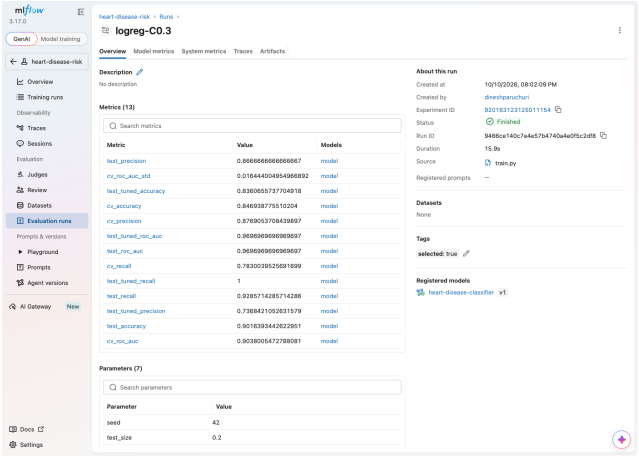


Figure 6 - MLflow run page of the selected run (metrics, parameters, registered model link)

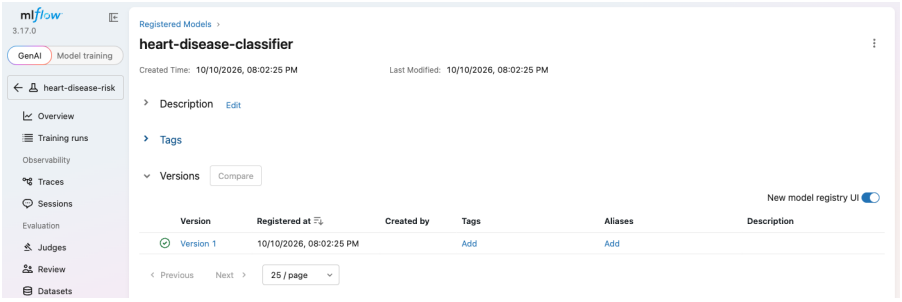


Figure 7 - model registry: heart-disease-classifier version 2

Model packaging and reproducibility

The deployable artifact is `models/model.joblib`, a dict holding the fitted *pipeline* (preprocessing + classifier), the decision *threshold* and the ordered *features* list. The same pipeline is also stored as an MLflow model inside each run. Pinned requirements, a fixed seed and the committed cleaned dataset make the whole chain reproducible from a clean checkout.

5. Architecture

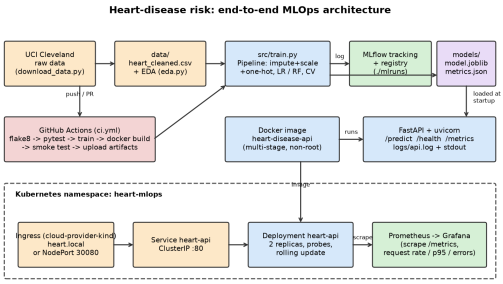


Figure 8 - architecture: training pipeline (top), CI and containerised serving (middle), Kubernetes deployment with Prometheus/Grafana monitoring (bottom)

How the pieces fit together. The UCI data is cleaned and explored before `src/train.py` evaluates the candidate models, logs the runs to MLflow and exports the winner. The GitHub Actions workflow is configured to lint, test, retrain, build the Docker image and smoke-test the API on pushes and pull requests. The container serves the saved model through FastAPI. In Kubernetes, two API replicas sit behind a Service, while Prometheus collects `/metrics` for the Grafana dashboard.

API contract.

Endpoint	Behaviour
POST /predict	13 raw features as JSON, validated by pydantic ranges (422 otherwise); returns prediction, label, confidence, probability_disease
GET /health	liveness/readiness; reports whether the model is loaded
GET /metrics	Prometheus exposition: api_requests_total, api_request_latency_seconds, model_predictions_total

Example request and response (captured in docs/run_outputs/api_samples.txt):

```
curl -X POST localhost:8000/predict -H 'Content-Type: application/json' \
```

```
  -d '{"age":57,"sex":1,"cp":4,"trestbps":140,"chol":241,"fbs":0,"restecg":0,
      "thalach":123,"exang":1,"oldpeak":0.2,"slope":2,"ca":0,"thal":7}'
{"prediction":1,"label":"heart disease","confidence":0.848,"probability_disease":0.848}
```

The API returns both the probability of the disease class and the confidence associated with the predicted label. For invalid input, such as an age of 250, FastAPI returns HTTP 422 and identifies the field that failed validation.

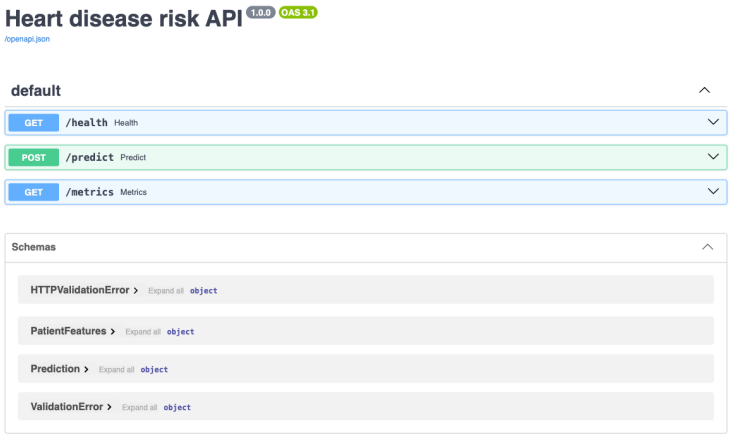


Figure 9 - FastAPI interactive documentation at /docs (real screenshot)

6. CI/CD pipeline and automated testing

Tests. 18 pytest tests (`18 passed, 1 warning in 0.10s`) cover: cleaning rules (question marks, label binarisation), the committed CSV's shape, the engineered feature, preprocessor output (no NaN, standardised numerics), robustness to unseen categories and missing values, the API (health, valid prediction, 7 out-of-range cases and a missing field returning 422, Prometheus counters) and a saved-model round trip. flake8 (max line length 100) reports no findings.

Workflow (`.github/workflows/ci.yml`, triggered on every push and pull request, ubuntu-latest, Python 3.11 with pip caching). Steps run in order and any non-zero exit fails the job; `set -o pipefail` with `tee` keeps logs while preserving failures:

#	Step	What it enforces
1	Install dependencies	clean install from requirements.txt
2	Lint	flake8 src test -> ci-output/flake8.txt
3	Unit tests	pytest -v + JUnit XML -> ci-output/
4	Train model	python -m src.train (9 MLflow runs) -> ci-output/train.txt
5	Verify artifacts	models/model.joblib and metrics.json must exist
6	Summarise MLflow runs	lists run metadata -> ci-output/mlruns-summary.txt
7	Build Docker image	docker build -t heart-disease-api:ci and :latest .
8	Smoke test container	scripts/smoke_test.sh: /health, /predict, /metrics
9	Kubernetes Ingress	disposable kind cluster; route all smoke checks via heart.local
10	Container logs	always(): docker logs -> ci-output/container.log
11	Upload artifacts	always(): model, metrics, logs/, ci-output/, mlruns/

Retraining in CI is intended to catch changes that break the training process, saved-model contract or container before a change is merged. Each workflow run also uploads logs, JUnit output, the model and the MLflow run store for inspection. The GitHub Actions workflow additionally creates a disposable kind cluster, installs the project manifests and tests the API through the Ingress host route. That cluster is deleted at the end of the job; it is not a persistent deployment.

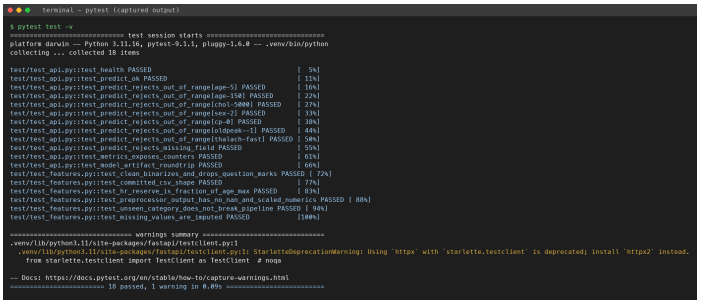


Figure 10 - pytest -v output (text captured from a real run, rendered to PNG; docs/run_outputs/pytest_verbose.txt)

Local and hosted CI evidence. `scripts/run_ci_local.sh` executes the prior workflow with `act` and local stubs for GitHub-only actions; its captured run passed 18 tests, lint, training, Docker build and container smoke tests. GitHub Actions run #4 passed all hosted steps, including the kind Ingress check; run #5 repeated the full workflow successfully on a subsequent documentation-only commit. Local act log: docs/run_outputs/act_ci_full.txt.



Figure 11 - GitHub Actions run #4 Ingress route and API smoke-test output; actual workflow artifact rendered locally

7. Containerisation

The `Dockerfile` is multi-stage: a builder stage creates a virtualenv and installs `requirements.txt`; the runtime stage (python:3.11-slim) copies only the venv, `src/` and `models/model.joblib`, so the image ships no training data or MLflow store. It runs as a non-root user (uid 10001), exposes port 8000, defines a HEALTHCHECK against `/health`, and starts `uvicorn src.api:app`. A build-time check fails the build if `src/api.py` is missing.

```
docker build -t heart-disease-api:latest .
docker run -d --name heart-api -p 8000:8000 heart-disease-api:latest
scripts/smoke_test.sh http://localhost:8000      # /health, /predict, /metrics
docker compose up --build                        # API + Prometheus :9090 + Grafana :3000
```

Local verification recorded for this project. The captured run shows that the image built successfully and the container started: `docker ps` reported *healthy* (HEALTHCHECK), `id -u` returned 10001 (non-root), `models/model.joblib` is present in the image, /health and /predict returned 200 and an invalid body returned 422. Output: docs/run_outputs/docker_run.txt.

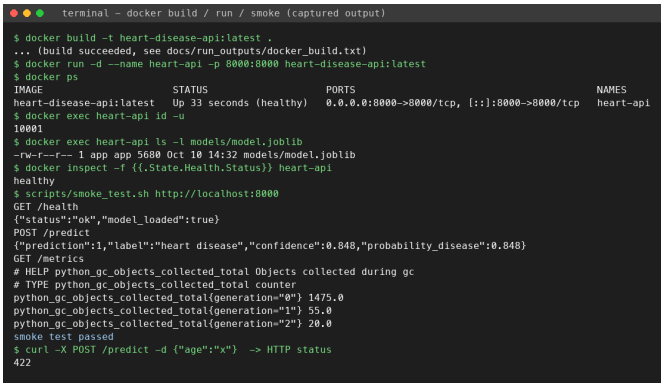


Figure 12 - docker run, health status, non-root user, smoke test and a 422 (real captured text)

8. Kubernetes deployment and monitoring

The manifests in `k8s/` are set up for a local kind or Minikube cluster. For a cloud cluster, the Service can be switched to a LoadBalancer and the Ingress controller configured for that environment.

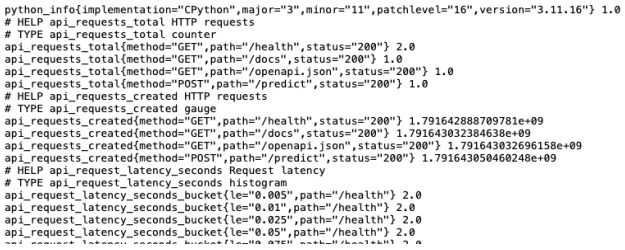
Manifest	Content
namespace.yaml	namespace heart-mlops

Manifest	Content
deployment.yaml	heart-api: 2 replicas, RollingUpdate (maxSurge 1, maxUnavailable 0), requests 100m/128Mi, limits 500m/512Mi, liveness + readiness probes on /health, Prometheus scrape annotations
service.yaml	ClusterIP service heart-api, port 80 -> container port 8000
ingress.yaml	classless Ingress, host heart.local -> heart-api (cloud-provider-kind)
service-nodeport.yaml	NodePort 30080 alternative (change type to LoadBalancer on a cloud cluster)
monitoring/	Prometheus (config map, RBAC, deployment, service), Grafana (datasource and dashboard provisioning), dashboard JSON

```
scripts/deploy_local.sh        # builds image, loads into kind/minikube, applies manifests
kubectl -n heart-mlops get pods,svc
kubectl -n heart-mlops port-forward svc/heart-api 8000:80
scripts/smoke_test.sh http://localhost:8000
```

Monitoring and logging

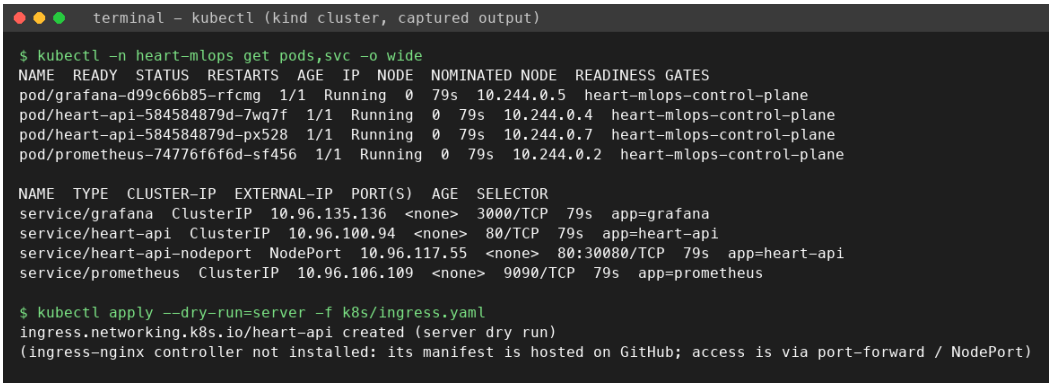
Logging. A middleware logs every request (method, path, status, latency in ms) to stdout - what `kubectl logs` and Docker show - and to `logs/api.log`. Model load and the threshold are logged at start-up. **Metrics.** Prometheus counters and histograms are exposed at `/metrics`: request count by method/path/status, latency histogram by path, and predictions by label. The provisioned Grafana dashboard 'Heart Disease API' has four panels: request rate, p95 latency, 5xx error share and predictions by label. The scrape target and dashboards are the same in docker-compose and Kubernetes.



```
python_info{implementation="CPython",major="3",minor="11",patchlevel="16",version="3.11.16"} 1.0
# HELP api_requests_total HTTP requests
# TYPE api_requests_total counter
api_requests_total{method="GET",path="/health",status="200"} 2.0
api_requests_total{method="GET",path="/docs",status="200"} 1.0
api_requests_total{method="GET",path="/openapi.json",status="200"} 1.0
api_requests_total{method="POST",path="/predict",status="200"} 1.0
# HELP api_requests_created HTTP requests
# TYPE api_requests_created gauge
api_requests_created{method="GET",path="/health",status="200"} 1.791642888709781e+09
api_requests_created{method="GET",path="/docs",status="200"} 1.791643032384638e+09
api_requests_created{method="GET",path="/openapi.json",status="200"} 1.791643032696158e+09
api_requests_created{method="POST",path="/predict",status="200"} 1.791643050460248e+09
# HELP api_request_latency_seconds Request latency
# TYPE api_request_latency_seconds histogram
api_request_latency_seconds_bucket{le="0.005",path="/health"} 2.0
api_request_latency_seconds_bucket{le="0.01",path="/health"} 2.0
api_request_latency_seconds_bucket{le="0.025",path="/health"} 2.0
api_request_latency_seconds_bucket{le="0.05",path="/health"} 2.0
```

Figure 13 - GET /metrics from the running API (api_requests_total and latency histogram visible; real screenshot)

Local Kubernetes evidence. The captured kind-cluster run (`scripts/deploy_local.sh`) shows 2/2 heart-api pods Ready, plus Prometheus and Grafana; the smoke test passed through `kubectl port-forward`; Prometheus showed both heart-api pods as *up*, and the Grafana dashboard loaded with data. In addition, the GitHub Actions workflow tests the Ingress route in a disposable kind cluster using cloud-provider-kind. This confirms routing for the CI environment only; no persistent public service is deployed. The older local screenshot below shows the port-forward run before that provider was added.



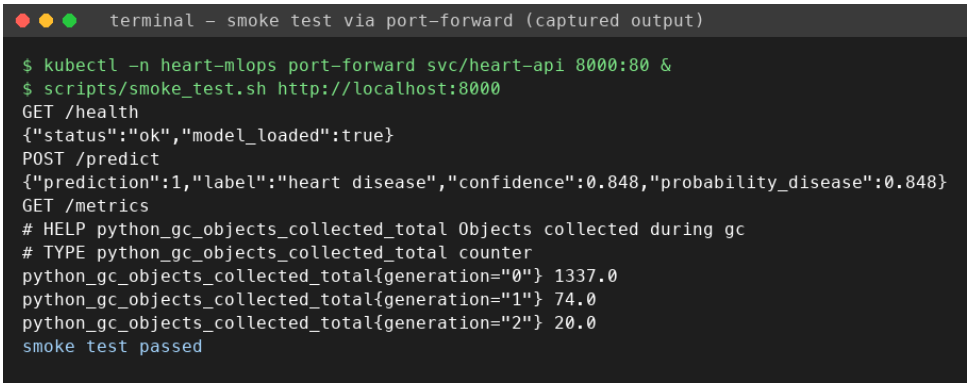
```
terminal — kubectl (kind cluster, captured output)

$ kubectl -n heart-mlops get pods,svc -o wide
NAME  READY  STATUS  RESTARTS  AGE  IP  NODE  NOMINATED NODE  READINESS GATES
pod/grafana-d99c66b85-rfcmg  1/1  Running  0  79s  10.244.0.5  heart-mlops-control-plane
pod/heart-api-584584879d-7wq7f  1/1  Running  0  79s  10.244.0.4  heart-mlops-control-plane
pod/heart-api-584584879d-px528  1/1  Running  0  79s  10.244.0.7  heart-mlops-control-plane
pod/prometheus-74776f6f6d-sf456  1/1  Running  0  79s  10.244.0.2  heart-mlops-control-plane

NAME  TYPE  CLUSTER-IP  EXTERNAL-IP  PORT(S)  AGE  SELECTOR
service/grafana  ClusterIP  10.96.135.136  <none>  3000/TCP  79s  app=grafana
service/heart-api  ClusterIP  10.96.100.94  <none>  80/TCP  79s  app=heart-api
service/heart-api-nodeport  NodePort  10.96.117.55  <none>  80:30080/TCP  79s  app=heart-api
service/prometheus  ClusterIP  10.96.106.109  <none>  9090/TCP  79s  app=prometheus

$ kubectl apply --dry-run=server -f k8s/ingress.yaml
ingress.networking.k8s.io/heart-api created (server dry run)
(ingress-nginx controller not installed: its manifest is hosted on GitHub; access is via port-forward / NodePort)
```

Figure 14 - kubectl get pods,svc (real captured text)



```
terminal — smoke test via port-forward (captured output)

$ kubectl -n heart-mlops port-forward svc/heart-api 8000:80 &
$ scripts/smoke_test.sh http://localhost:8000
GET /health
{"status":"ok","model_loaded":true}
POST /predict
{"prediction":1,"label":"heart disease","confidence":0.848,"probability_disease":0.848}
GET /metrics
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 1337.0
python_gc_objects_collected_total{generation="1"} 74.0
python_gc_objects_collected_total{generation="2"} 20.0
smoke test passed
```

Figure 15 - smoke test served by the pods through port-forward (real captured text)

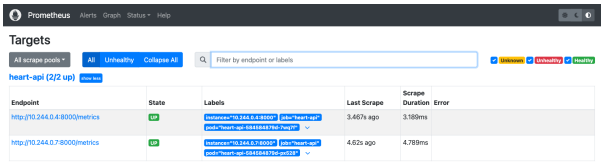


Figure 16 - Prometheus targets page (real browser screenshot)

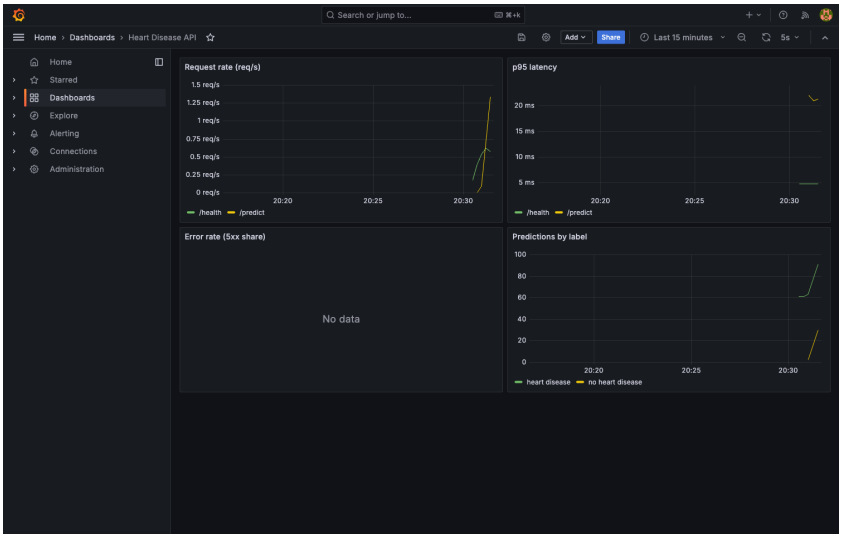


Figure 17 - Grafana 'Heart Disease API' dashboard (real browser screenshot; the 5xx panel is empty because no errors occurred)

9. Results and limitations

Results

- Best model: logreg-C0.3 - CV ROC-AUC 0.904, CV accuracy 0.847, CV recall 0.783.
- Held-out test at the default 0.5: accuracy 0.902, precision 0.867, recall 0.929, ROC-AUC 0.970.
- At the recall-oriented threshold 0.26: accuracy 0.836, precision 0.737, recall 1.000.
- Random forests reached CV ROC-AUC 0.876-0.895; none beat logistic regression.
- 18 unit tests pass, flake8 is clean, and the API returns validated predictions with request logging and Prometheus metrics.

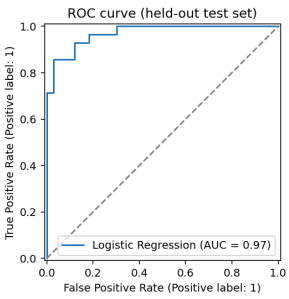


Figure 18 - ROC curve on the held-out test set

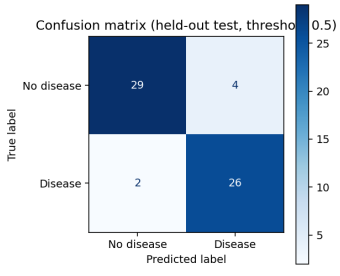


Figure 19 - confusion matrix at the default threshold (0.5)

Limitations and honest caveats

- **Small data.** 303 patients (61 test rows): one patient moves a metric by ~1.6 points, and CV standard deviations (up to 0.037) exceed many gaps between configurations. Treat the ranking of close runs as noise.

- **Single hold-out split.** The threshold and model were chosen with training-only data, but a single split cannot quantify variance; repeated CV or an external cohort would be stronger.
- **Population shift.** The Cleveland cohort is one hospital from the 1980s; calibration and performance on other populations are unknown. This is a teaching project and not a medical device.
- **Precision trade-off.** The 0.26 threshold maximises recall and therefore produces more false positives.
- **Deployment verification.** Local Docker and kind deployment evidence is included. GitHub Actions runs #4 and #5 passed the full application pipeline and verified Ingress routing in a disposable CI cluster. No persistent public API endpoint is deployed.
- **Monitoring scope.** Operational metrics only: no data-drift or prediction-distribution monitoring and no automated retraining trigger. The MLflow store is local files, so it is not shared between machines.

Possible next steps: drift monitoring on input features, a remote MLflow tracking server, image push to a registry from CI, and a Helm chart for the manifests.

10. Repository and deliverables map

Code repository: https://github.com/dineshsaitejaparuchuri/mlops-assignment **API access:** no public endpoint is configured; run locally (`uvicorn src.api:app --port 8000`, `docker run -p 8000:8000 heart-disease-api`, or the Kubernetes port-forward above) and open http://localhost:8000/docs.

Assignment requirement	Where it is
EDA, cleaning, download script	src/download_data.py, src/eda.py, notebooks/01_eda.ipynb, docs/eda_summary.txt, Section 2
Features and models (>=2, CV, metrics)	src/features.py, src/train.py, notebooks/02_training.ipynb, docs/experiments.md, Section 3
Experiment tracking	mlruns/ (via src/train.py), screenshots/mlflow_*.png, Section 4
Packaging and reproducibility	models/model.joblib, requirements.txt, pipeline in src/features.py
Unit tests + CI/CD	test/, .github/workflows/ci.yml, docs/run_outputs/, Section 6
Docker	Dockerfile, docker-compose.yml, scripts/smoke_test.sh, Section 7
Kubernetes	k8s/, scripts/deploy_local.sh, Section 8
Monitoring and logging	src/api.py, monitoring/, k8s/monitoring/, Section 8
Inference notebook	notebooks/03_inference.ipynb
Written report	docs/report/ (this PDF)
Demo video outline	docs/demo_video_script.md (recording not included in this package)

11. Evidence screenshots

The `screenshots/` folder contains EDA plots, MLflow pages, API documentation and sample responses, Prometheus/Grafana views, plus captured test, lint, Docker, Kubernetes and local-CI output. Browser images were captured from local services; terminal images were rendered from saved command output. Hosted CI status is available in the repository Actions tab. The Ingress route is tested in an ephemeral cluster; this does not provide a public endpoint.